CE 459: VLSI Algorithmics

Lab 3. Loop unrolling and pipelining

Disclaimer: This tutorial was based on original ***Mentor Graphics*** tutorials. Slight modifications were required due to divergences in the versions of the tools and deprecated instructions. All credits go to the authors of the original tutorials.

In this tutorial we will inspect the following topics:

- (1) Use loop unrolling and loop pipelining to improve design performance
- (2) Analyze designs to see the effects of loop unrolling and bad coding style
- (3) Recode the C++ to improve area and runtime

Part 1 of the lab shows how to understand the effects of loop unrolling and pipelining as well as how to analyze the effects of "bad" coding style for HLS when combined with loop unrolling. It shows how unrolling loops with operations such as conditional accumulates can create long dependency chains of logic.

Part 2 of this lab will look at the effects of dynamically indexing an array mapped to registers inside a loop that is unrolled.

Before we begin with the tutorial...

Remember to check Tutorial 0 to assert how to setup ssh to access the Wilkinson server and configure and run Catapult for the first time. The link is here.

Part 1 - Steps:

0. Access the Wilkinson server with your NetID and open a Terminal.

1. Load the cad tools environment by running command **(1)**:

```
>> source ~/envsetup-catapult.sh
```
(1)

2. Navigate to the Labs directory and then to *Lab3/conditional_acc* using command **(2)**:

```
>> cd ~/Labs/Lab3/conditional_acc
```
(2)

3. Before running catapult, let's talk a bit about tcl files. A tcl file is a file containing some instructions which are specific to a certain app. Catapult has an inner console, on which we are able to execute different instructions one by one, instructions such as "create new project", "read project sources", "synthesize", etc. Instead of having to perform these common actions one by one by hand, we automate this process by creating a tcl file and sourcing it inside catapult. This will run a series of commands and setup our project automatically. Before actually running it let's take a look at the tcl file we are going to source. Open *"directives.tcl"* with your favorite text editor (vim/gedit,...). The content should match the one shown in **Figure 1**.

   The "solution" directives basically setup our project, project sources, etc. The "directive set" instruction sets some of the parameters. First of all, we will need to comment line #10 "*directive set -OLD_SCHED false*", which includes directive "*OLD_SCHED*" which has been deprecated in our version of catapult. Save the document and close it.

mbv6231@batgirl:conditional_acc ⌥⌘1

```
1 #Lab3 - conditional acc
2 solution new -state initial
3 solution options defaults
4 solution options set /Output/GenerateCycleNetlist false
5 solution options set /Flows/SCVerify/USE_CCS_BLOCK true
6 solution file add ./test_orig.cpp -type C++
7 solution file add ./test.cpp -type C++ -exclude true
8 solution file add ./tb.cpp -type C++ -exclude true
9 directive set -DESIGN_GOAL area
10 #directive set -OLD_SCHED false
11 directive set -SPECULATE true
12 directive set -MERGEABLE true
13 directive set -REGISTER_THRESHOLD 256
14 directive set -MEM_MAP_THRESHOLD 32
15 directive set -LOGIC_OPT false
16 directive set -FSM_ENCODING none
17 directive set -FSM_BINARY_ENCODING_THRESHOLD 64
18 directive set -REG_MAX_FANOUT 0
19 directive set -NO_X_ASSIGNMENTS true
```

Figure 1. Content of the file *"directives.tcl"*.

4. Let's launch Catapult now while being in this directory by running **(3)**:

```
>> catapult -product ultra
```
(3)

5. Go to **File** → **Run** script and select "*directives.tcl*" file, as shown in **Figure 2**. This will:
    a. Add input files
    b. 2. Set the synthesis tool to OasysRTL
    c. 3. Set the clock frequency to 1000MHz

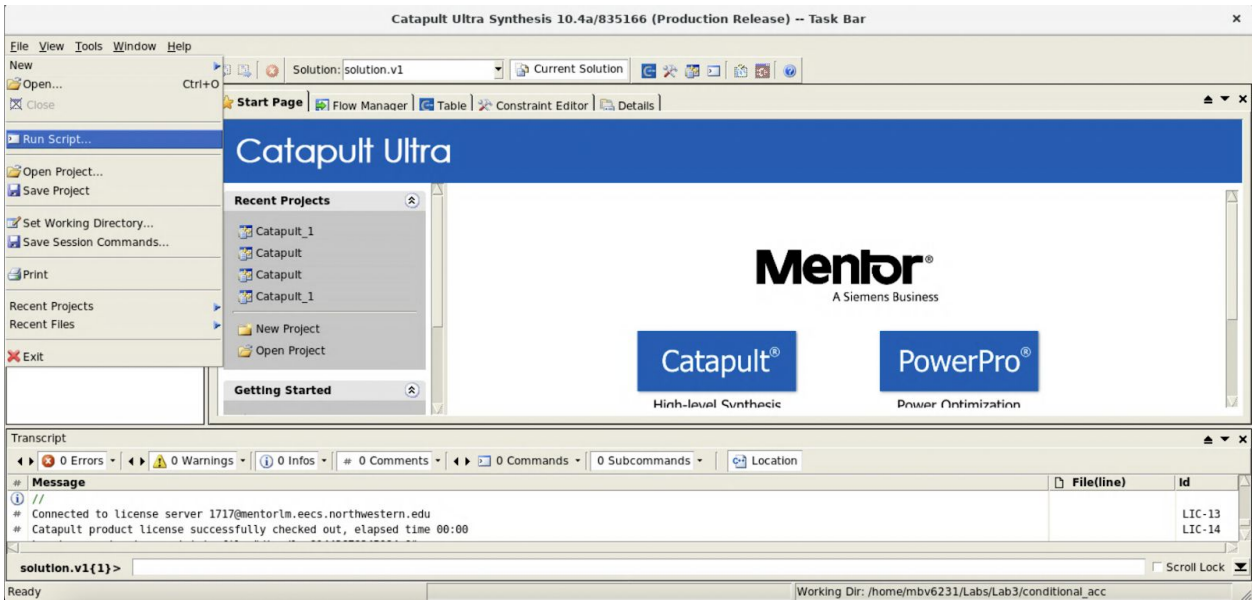


Figure 2. Catapult run script menu.

6. Double click on the "*test_orig.cpp*" file as shown in the Projects file pan on the left (**Figure 3**). This will open the source code in a new tab in the main catapult panel.

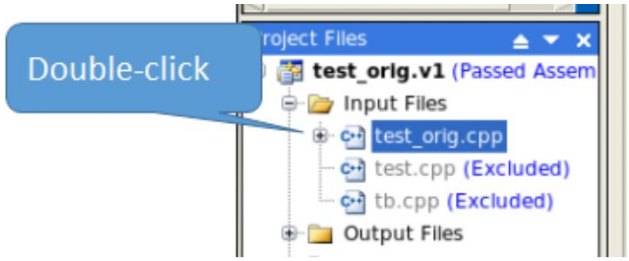
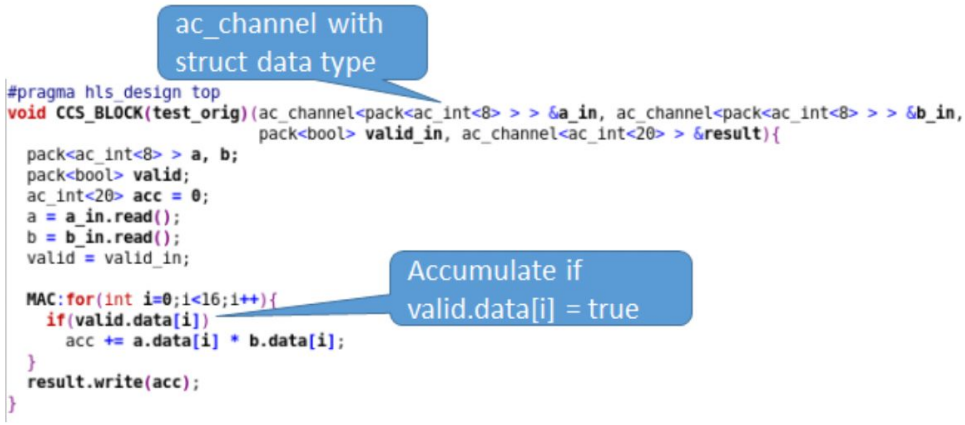


Figure 3. Catapult run script menu.

7. Examine the C++ design and note the following:
    a. The design has *ac_channels* for *a_in* and *b_in* that have a struct data type called "*pack*".

b. An interface variable "*valid_in*" is used to conditionally multiply-accumulate 16 values from *a_in* and *b_in*.




```
#pragma hls_design top
void CCS_BLOCK(test_orig)(ac_channel<pack<ac_int<8> > > &a_in, ac_channel<pack<ac_int<8> > > &b_in,
                          pack<bool> valid_in, ac_channel<ac_int<20> > &result){
  pack<ac_int<8> > a, b;
  pack<bool> valid;
  ac_int<20> acc = 0;
  a = a_in.read();
  b = b_in.read();
  valid = valid_in;

  MAC:for(int i=0;i<16;i++){
    if(valid.data[i])
      acc += a.data[i] * b.data[i];
  }
  result.write(acc);
}
```

Figure 4. Content of file "*test_orig.cpp*".

8. Double-click on the **#include "test.h"** at the top of the "*test_orig.cpp*" file and note that the "pack" data type is defined here and contains an array of 16 words.

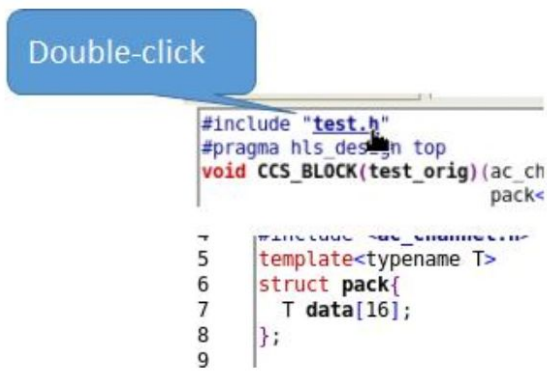


Figure 5. Content of file "*test.h*".

9. Click on Architecture in the Task Bar and expand the Interface Folder. Note that "a_in" and "b_in" interfaces are 16x8 or 128 bits wide. All 16 values are available in parallel.

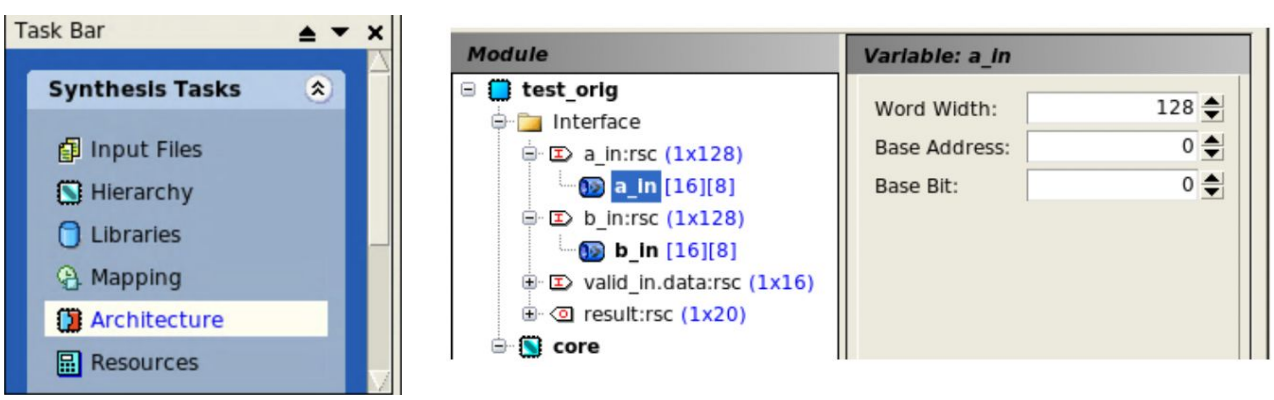


Figure 6. (Left) Task bar. (Right) Module interfaces detail.

10. Click on Schedule in the Task bar. Look at the table view and note that the latency is 33 and throughput is 34.

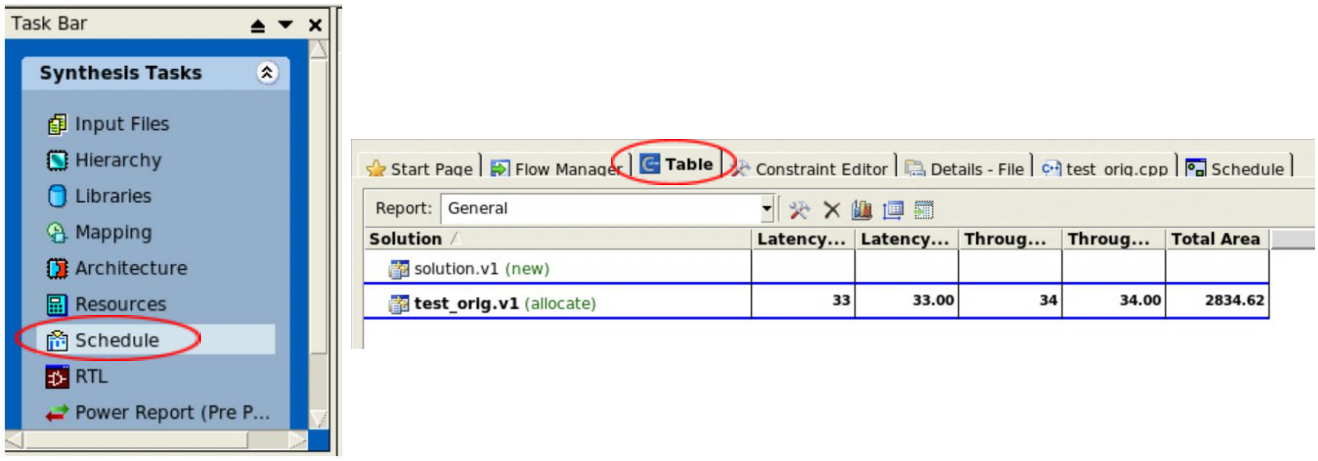


Figure 7. (Left) Schedule menu entry in Task Bar. (Right) Table view detail.

11. Open the Catapult Gantt chart (by clicking back into "Schedule" in the Table View) and expand all loops by clicking on the icon with the down arrow in the tool bar.

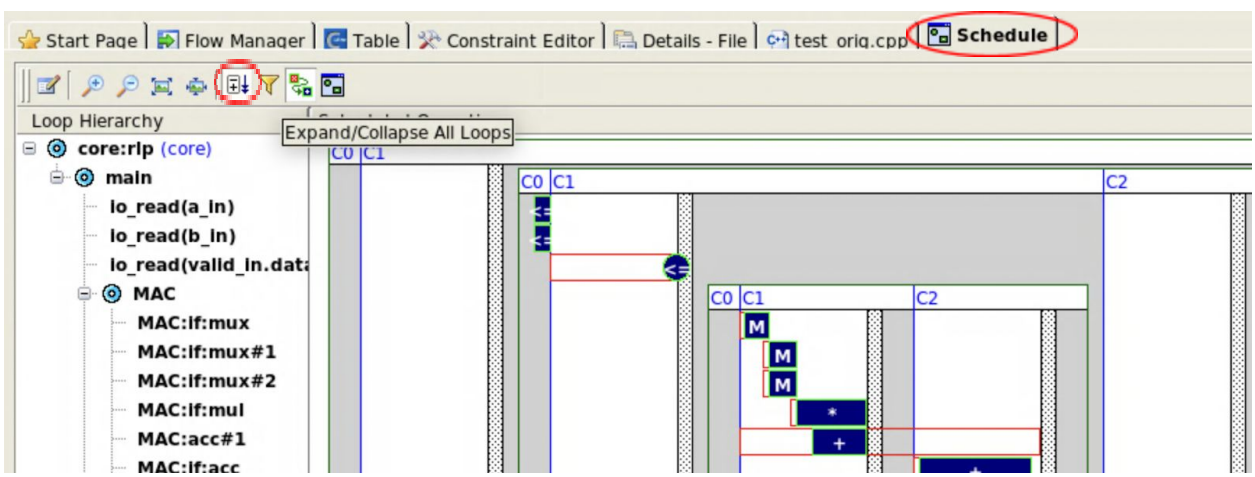


Figure 8. Gantt view (Schedule tab) and expansion of loops.

12. Look at the Gantt chart and note the following:
    a. The read of *a_in* and *b_in* is taking one c-step (cycle)
    b. The write of result takes 1 c-step
    c. The MAC loop body has 2 c-steps
    d. Most of the runtime (Big green bar) is spent executing the MAC loop
    e. Hover the mouse over the MAC loop and see that it takes 32-cycles to execute. This is because the unpipelined loop has 16 iterations with each iteration taking 2 c-steps. 16x2 = 32.

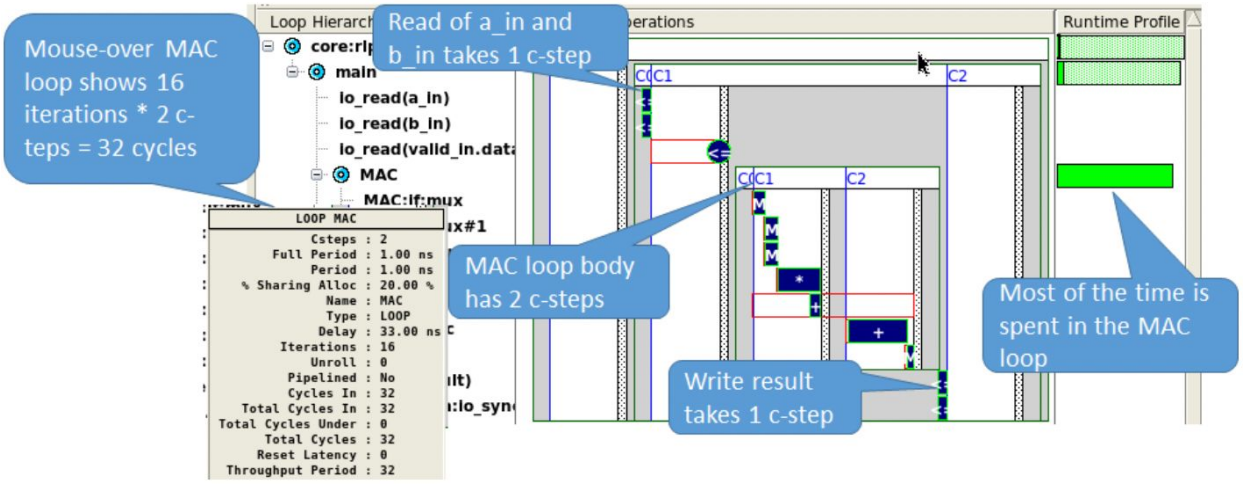


Figure 9. Detail of the gantt chart.

13. Click on RTL in the Task Bar to generate RTL.

14. Click on Architecture in the Task Bar.

15. Select the MAC loop and set the Pipeline Initiation Interval to II=1. Click *Apply*.

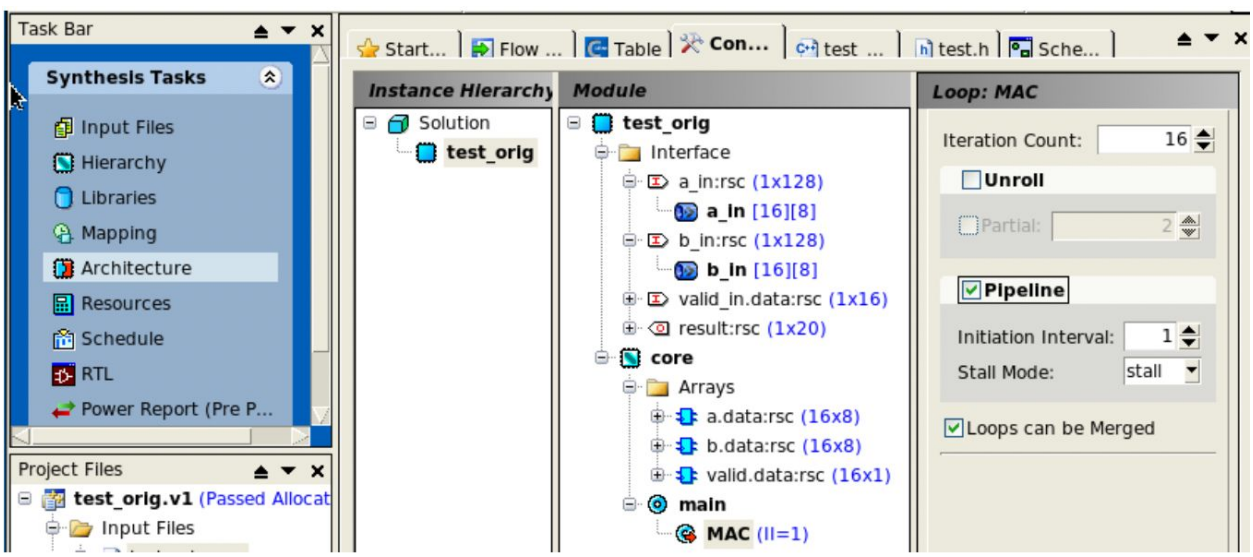
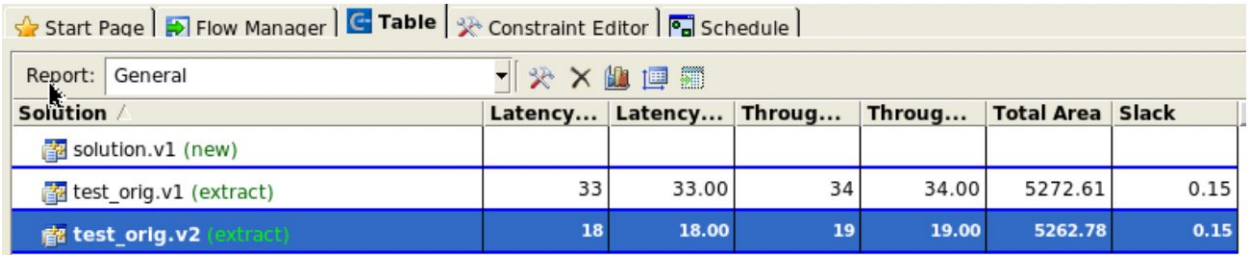


Figure 10. Setting the II to 1 in the MAC loop.

16. Click on RTL in the Task Bar to synthesize with the new configuration (II = 1).

17. Look at the Table View and note that the latency has been reduced to 18 and the throughput to 19. Also, note that the area has gotten slightly smaller. Pipelining can often improve performance with less of an impact on area.



Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
solution.v1 (new)						
test_orig.v1 (extract)	33	33.00	34	34.00	5272.61	0.15
test_orig.v2 (extract)	**18**	**18.00**	**19**	**19.00**	**5262.78**	**0.15**

Figure 11. Latency and throughput of design with II=1 in the MAC loop.

18. Click on back on Schedule in the Task Bar and open the Gantt Chart. Expand all loops in the Gantt chart and note that most of the time is still spent in the MAC loop (big green bar) but 2 c-steps are reading and writing the IO in the “main” loop (Little green bar).

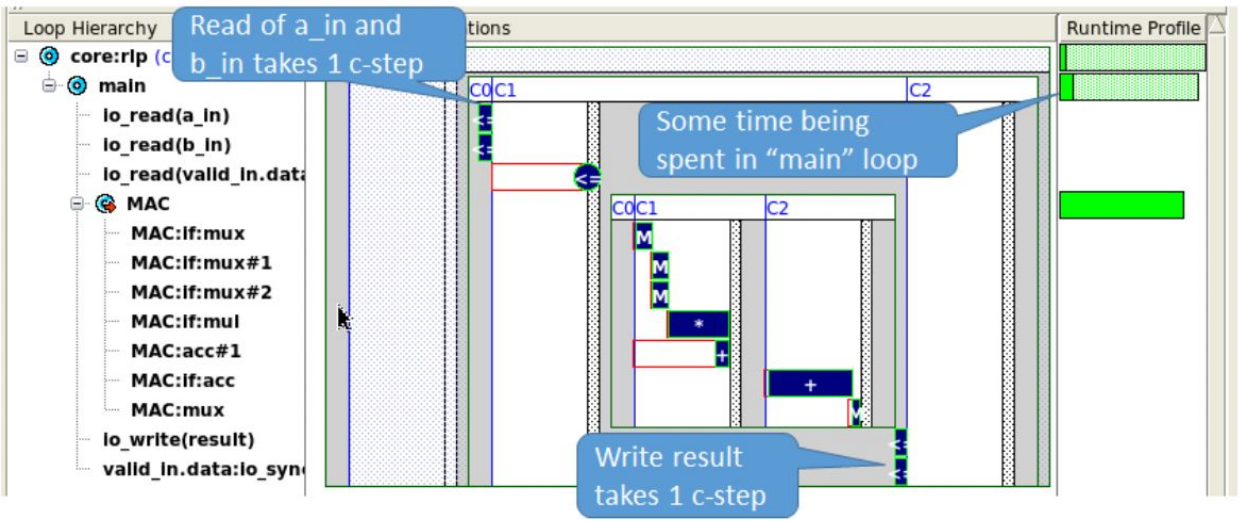
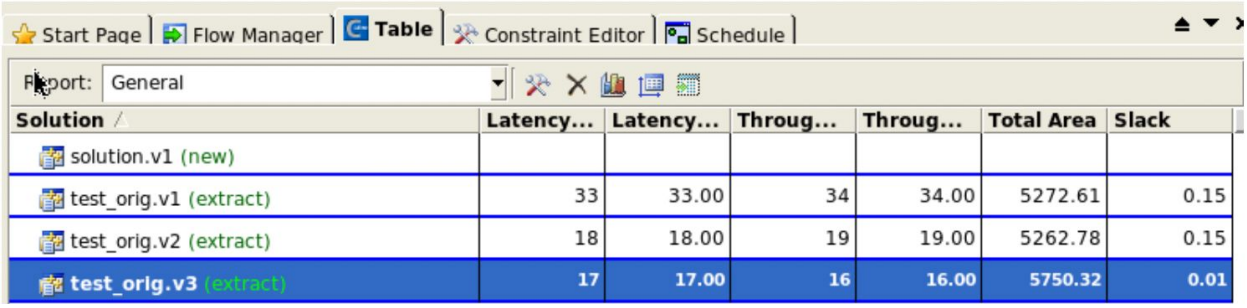


Figure 12. Detail of the time spent in the MAC loop, as shown in the Gantt chart in Catapult.

19. Click on Architecture in the Task Bar and pipeline the “main” loop with II=1. Click *Apply*.

20. Click on RTL in the Task Bar to synthesize with the new configuration.

21. Look at the Table View and Note:
    a. The latency is 17 and the throughput is 16
    b. The area has increased slightly.



Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
solution.v1 (new)						
test_orig.v1 (extract)	33	33.00	34	34.00	5272.61	0.15
test_orig.v2 (extract)	18	18.00	19	19.00	5262.78	0.15
test_orig.v3 (extract)	17	17.00	16	16.00	5750.32	0.01

Figure 13. Detail of the latency/throughput of the synthesized sources.

22. Click on Schedule in the Task Bar and open the Gantt chart. Note that the “main” loop and the MAC loop have been flattened together.

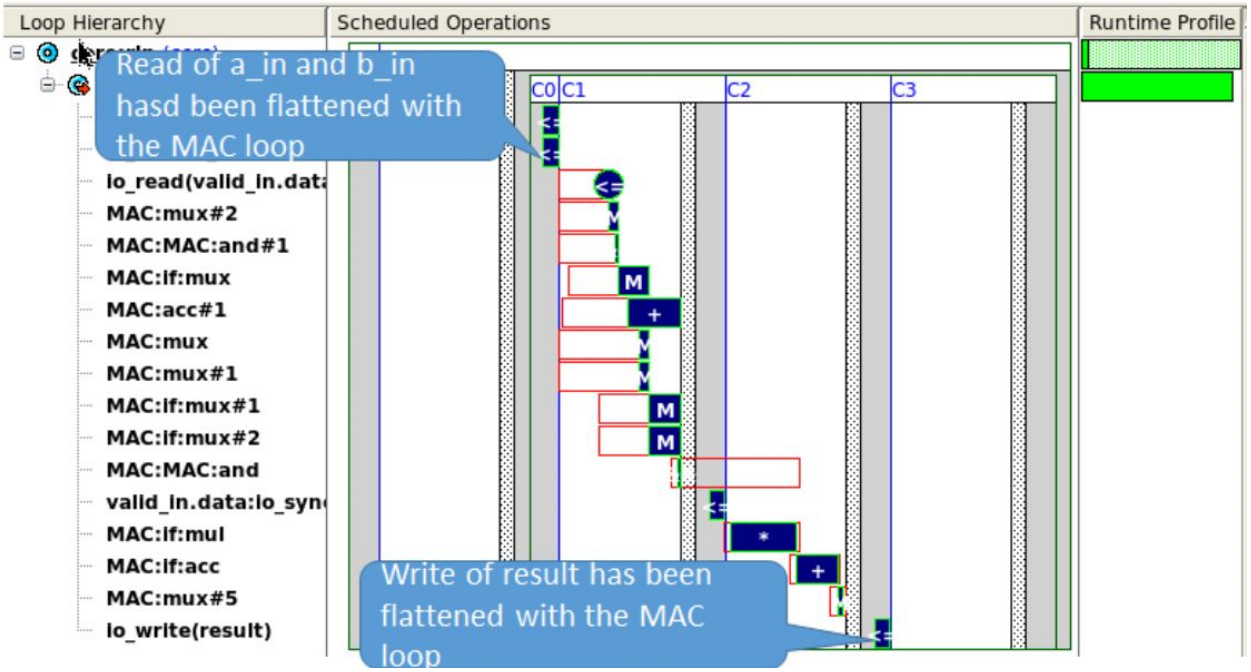
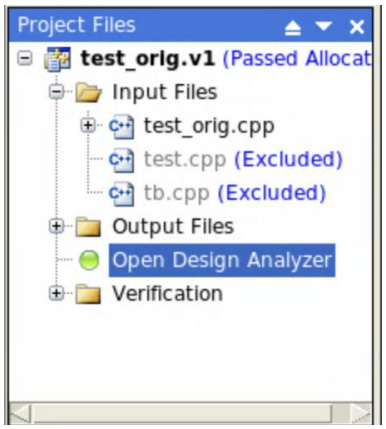


Figure 14. Detail of the Gantt chart.

23. Go to **Project Files** → **Open Design Analyzer** and launch Design Analyzer by double-clicking.



24. Switch the Design Analyzer Design Analyzer view state to extract and click on the type counts tab. This gives a summary of all hardware resources. Note that a single hardware multiplier is used for this design. Click to select the multiplier in the object list and note how it is highlighted in all other views.

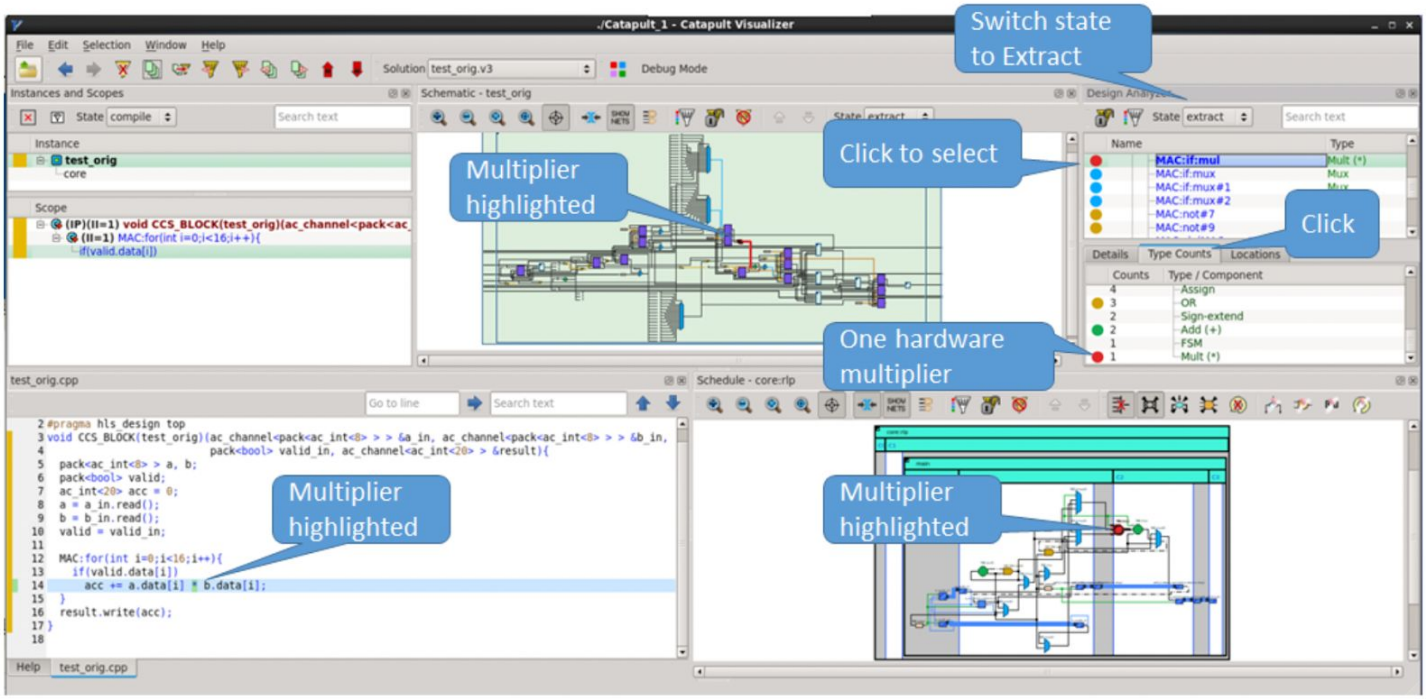


Figure 15. Detail of the design analyzer.

25. Close Design Analyzer.

26. In Catapult click on Architecture in the Task Bar.

27. Select the MAC loop and unroll partially by 2.

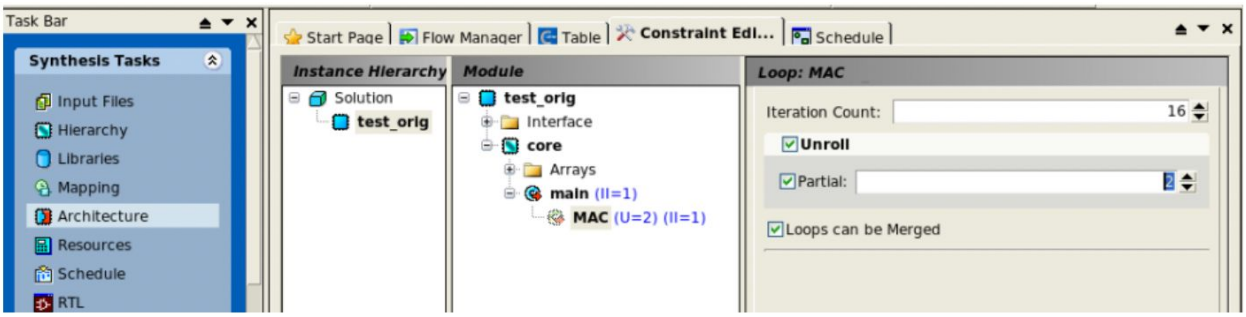


Figure 16. Detail of partially unrolling the MAC loop to 2.

28. Generate RTL.

29. Look at the table view and note that the throughput has been reduced to 8 cycles, with an increase in area.

Start Page | Flow Manager | Table | Constraint Editor

Report: General

Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
solution.v1 (new)						
test_orig.v1 (extract)	33	33.00	34	34.00	5272.61	0.15
test_orig.v2 (extract)	18	18.00	19	19.00	5262.78	0.15
test_orig.v3 (extract)	17	17.00	16	16.00	5750.32	0.01
test_orig.v4 (extract)	**9**	**9.00**	**8**	**8.00**	**6586.78**	**0.04**

Figure 17. Detail of table view.

30. Repeat steps 23 to 25 and verify that the design is now implemented using 2 hardware multipliers. Loop unrolling the MAC loop by 2x has doubled the number of hardware resources.

31. In Catapult click on Architecture in the Task Bar.

32. Select the MAC loop and fully unroll the loop (Unselecting the checkbox "*partial*" in MAC Loop dialog box shown in **Figure 16**).

33. Click on RTL in the Task Bar to resynthesize the sources with the fully unrolled loop.

34. Look at the Table View and note:
    a. The throughput is now equal to one but the latency is still 9
    b. The area has increased substantially.

Start Page | Flow Manager | Table | Constraint Editor

Report: General

Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
solution.v1 (new)						
test_orig.v1 (extract)	33	33.00	34	34.00	5272.61	0.15
test_orig.v2 (extract)	18	18.00	19	19.00	5262.78	0.15
test_orig.v3 (extract)	17	17.00	16	16.00	5750.32	0.01
test_orig.v4 (extract)	9	9.00	8	8.00	6586.78	0.04
test_orig.v5 (extract)	**9**	**9.00**	**1**	**1.00**	**19352.87**	**0.09**

Figure 18. Detail of table view.

35. Repeat steps 23 to 25 and verify that the design is now implemented using 16 hardware multipliers.

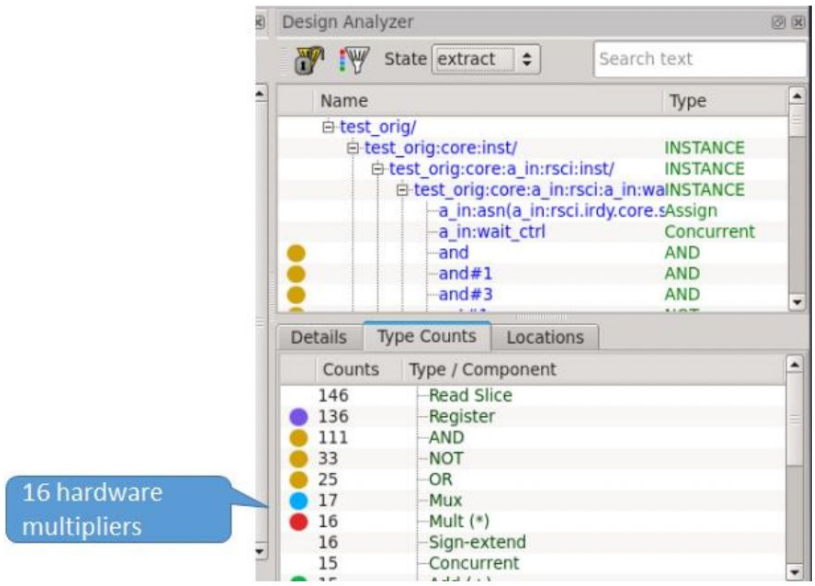


Figure 19. Detail of the Design view and number of multipliers in the Designer View.

36. Look at the Design Analyzer Schedule view and note:
    a. All the multipliers are scheduled in c-step 1
    b. There is a diagonal chain of operations (add and mux) scheduled in c-step 2 to c-step 9.

37. Click through the chain of adders and muxes in the Schedule view and see how the “if” condition and accumulate operation are highlighted in the C++.

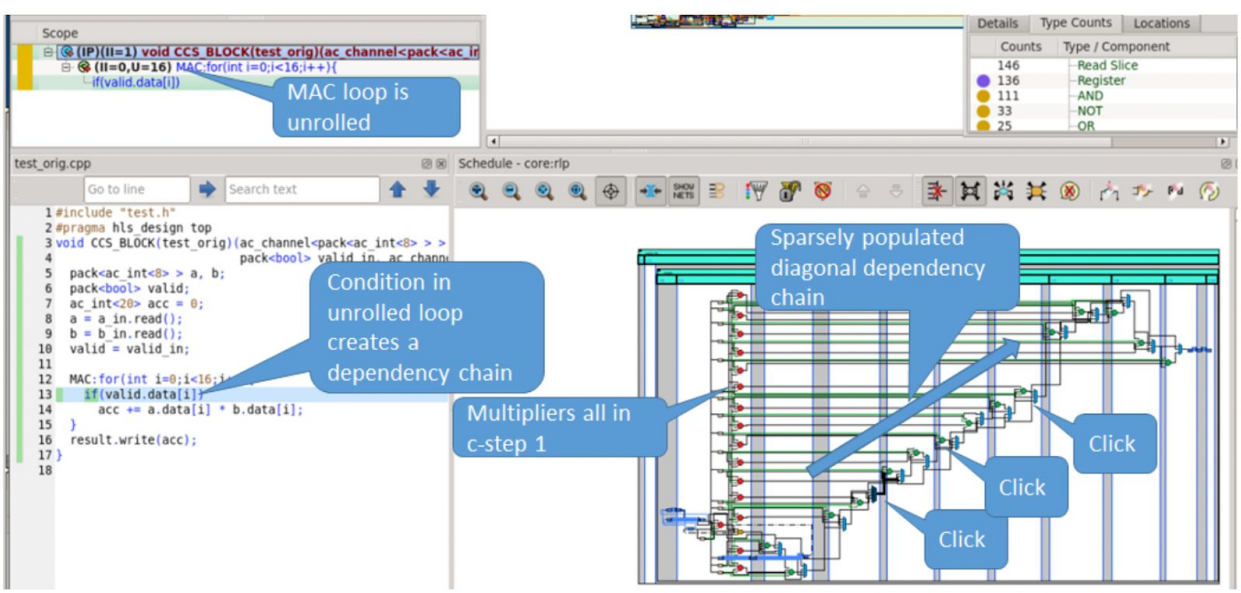


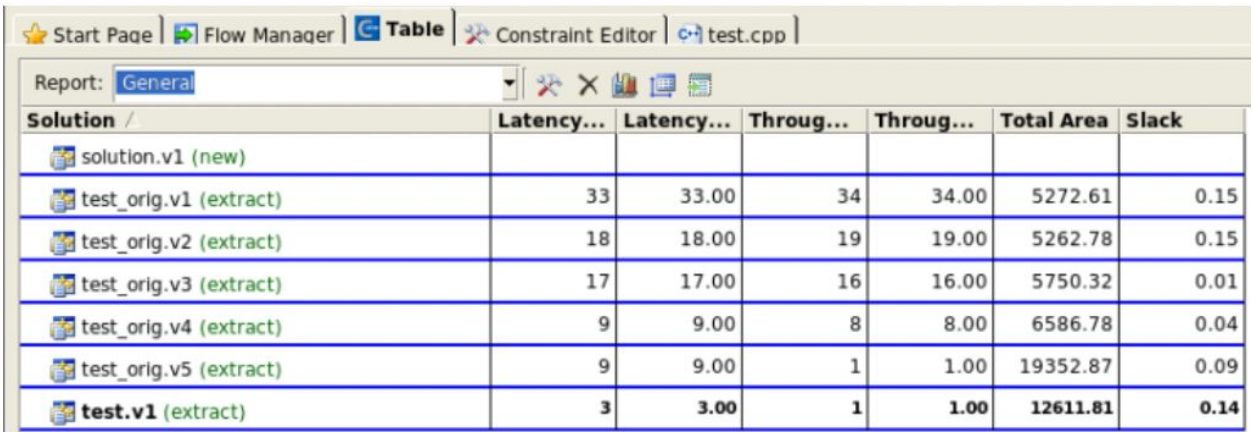
Figure 20. Detail of the Design view.

38. Unrolling loops with conditions, conditional breaks, or complex control may result in priority encoded dependency chain logic.

39. Close Design Analyzer.

40. Go to **File** → **Run** script and select "*coding_style.tcl*" file. This will make the "*test.cpp*" the top-level design, re-constraint the design, and generate RTL.

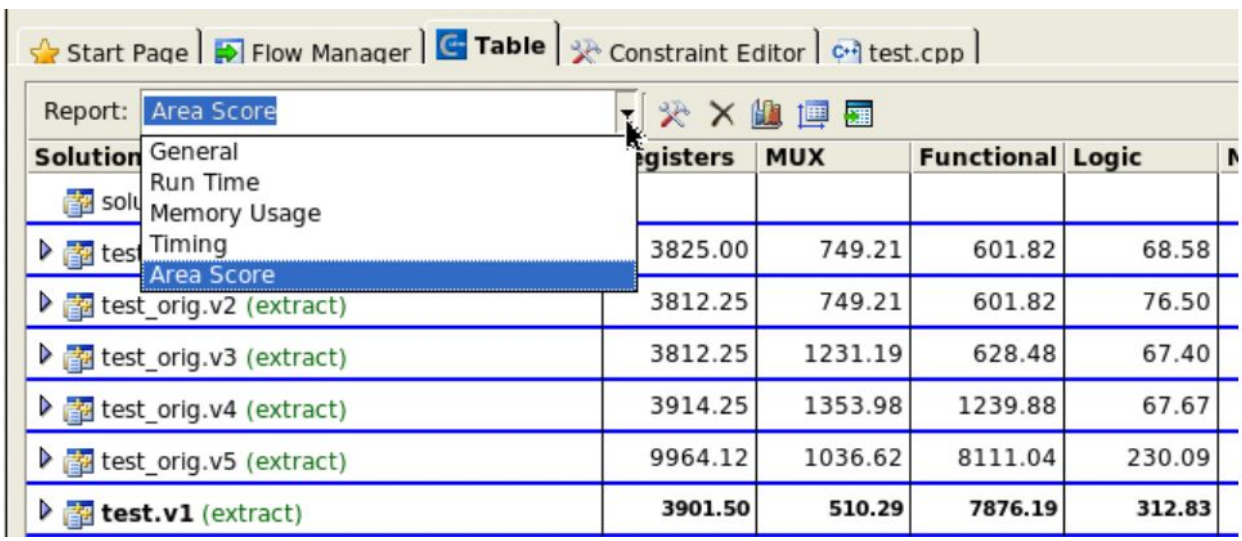
41. Look at the table view and Note that the latency is now 3, throughput is 1, and the area has gone down significantly.



Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
solution.v1 (new)						
test_orig.v1 (extract)	33	33.00	34	34.00	5272.61	0.15
test_orig.v2 (extract)	18	18.00	19	19.00	5262.78	0.15
test_orig.v3 (extract)	17	17.00	16	16.00	5750.32	0.01
test_orig.v4 (extract)	9	9.00	8	8.00	6586.78	0.04
test_orig.v5 (extract)	9	9.00	1	1.00	19352.87	0.09
test.v1 (extract)	**3**	**3.00**	**1**	**1.00**	**12611.81**	**0.14**

Figure 21. Detail of table view.

42. Change the Table View report to **Area Score** and note that the registers area has decreased by a large amount. This should make sense because the design latency went from 9 cycles to 3 cycles so there are fewer pipeline registers.



Solution	gisters	MUX	Functional	Logic
solu				
tes	3825.00	749.21	601.82	68.58
test_orig.v2 (extract)	3812.25	749.21	601.82	76.50
test_orig.v3 (extract)	3812.25	1231.19	628.48	67.40
test_orig.v4 (extract)	3914.25	1353.98	1239.88	67.67
test_orig.v5 (extract)	9964.12	1036.62	8111.04	230.09
test.v1 (extract)	**3901.50**	**510.29**	**7876.19**	**312.83**

Figure 22. Detail of table view switching to Area score.

43. Go to the Input Files Folder and double-click on "*test.cpp*", just like we did in step 6.

44. Note how the code has been rewritten to that there is no longer a conditional accumulate. The "*?*" operator is used to always add either the result from the multiplication or zero.

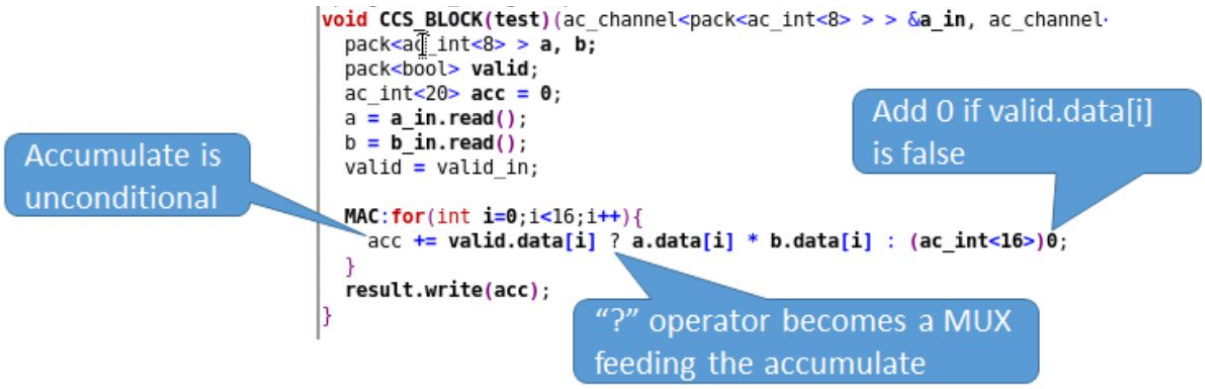


Figure 23. Detail of "*test.cpp*" file.

45. Open Catapult Design Analyzer and look at the hardware structure.

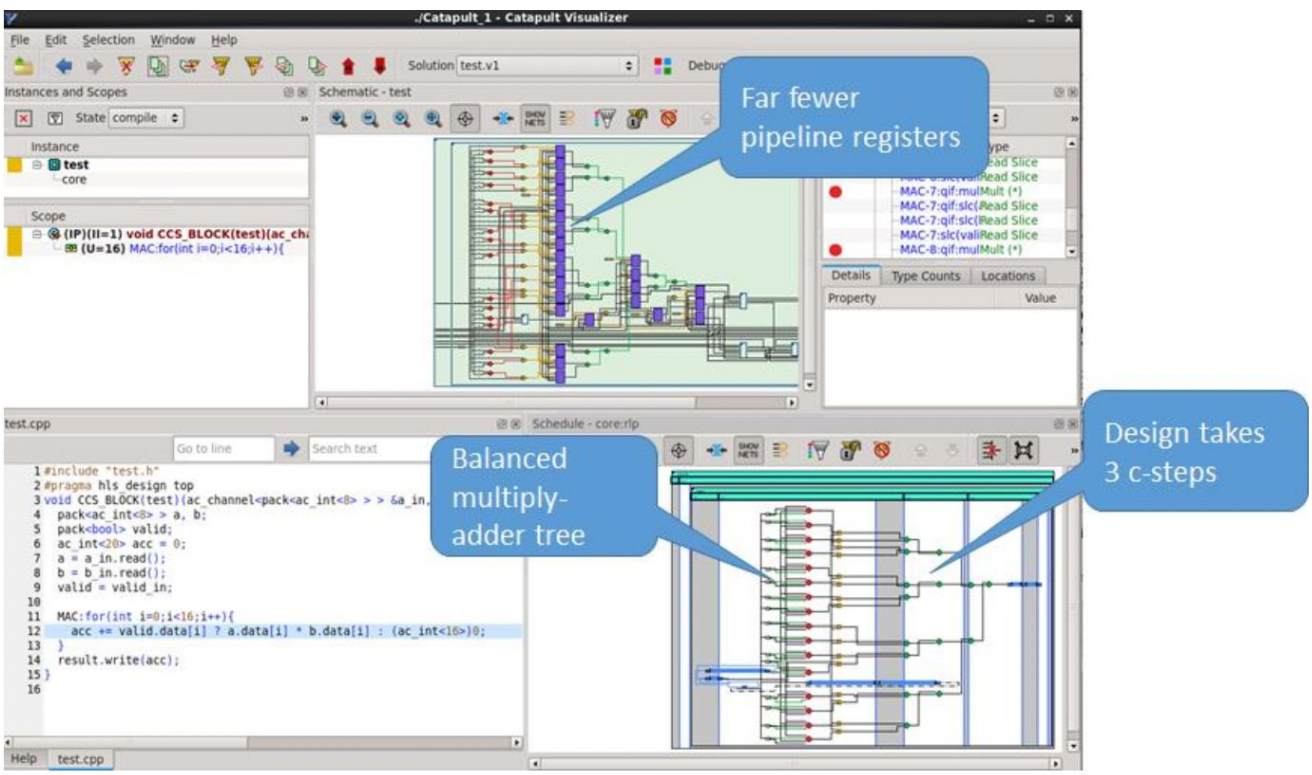


Figure 24. Detail of Design Viewer for the newly imported design.

46. Close Design Analyzer and Catapult altogether.

Part 2 - Steps:

1. Access the Wilkinson server with your NetID and open a Terminal.

2. Load the cad tools environment by running command **(1)**:

```
>> source ~/envsetup-catapult.sh
```
(1)

3. Navigate to the Labs directory and then to *Lab3/dynamic_index* using command **(2)**:

```
>> cd ~/Labs/Lab3/dynamic_index
```
(2)

4. Let's launch Catapult now while being in this directory by running **(3)**:

```
>> catapult -product ultra
```
(3)

5. Go to **File** → **Run** script and select "*directives.tcl*" file, as shown in **Figure 2**. This can take a couple of minutes to complete.

6. Open the design "*test_orig.cpp*" and look at the C++ code.
   a. This design copies an input array "*din[40]*" with 40 elements to an internal array "*regs[40]*" which is mapped to registers. The index to write "regs" contains a variable "*offset*" plus the loop index "I" to select where to begin writing.

   b. The loop, which has 40 iterations is fully unrolled. By unrolling the dynamic index "*regs[i+offset]*" we are creating 40 dynamic indices into "regs".

   c. Catapult will split an array mapped to registers with a dynamic index into many variables. If the array is inside an unrolled loop it will be split many times. This will lead to long runtime and larger area.

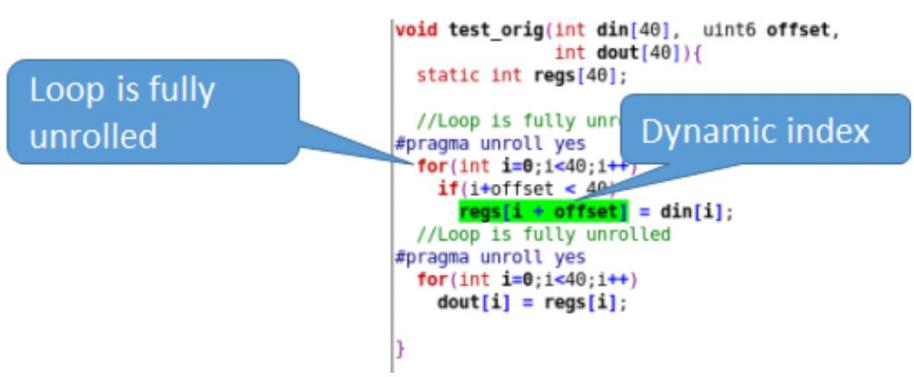


Figure 25. Detail of file "*test_orig.cpp*".

7. Go to the Catapult transcript and set the filter for Warnings to "*Show Warnings Only*".

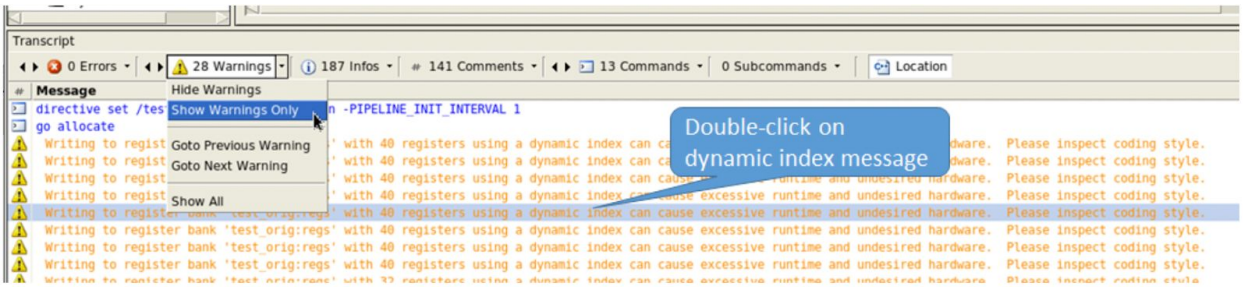


Figure 26. Detail of warnings.

8. Find the warnings about "*dynamic index*" and double-click on them to cross to the C++. The warnings are all cross-probing back to the line that indexes "*regs[i+offset]*".

```
7   #pragma unroll yes
8     for(int i=0;i<40;i++)
9       if(i+offset < 40)
10        regs[i + offset] = din[i];
11    //Loop is fully unrolled
12  #pragma unroll yes
```

Figure 27. Detail of the variable "*regs[i+offset]*" in file "*test_orig.cpp*".

9. Open Catapult Design Analyzer. Note it may take several seconds to load.

10. Click on the "*Locations*" tab in the Design Analyzer window. This provides an ordered (largest to smallest) list of the number of operations per line of C++. A very large number of operations associated to a single line of C++ is usually an indication that an optimization has "blow-up" the design.
    a. Note that there are thousands of operations (and,readslice,etc) all coming from a single line of C++.
    b. Click on line in the Locations tab and it will highlight the C++ source as well as the scope where the source comes from.
    c. Note that the source contains a dynamic index.
    d. Note that the highlighted scope in the scope view is inside of a loop that is unrolled 40 times.

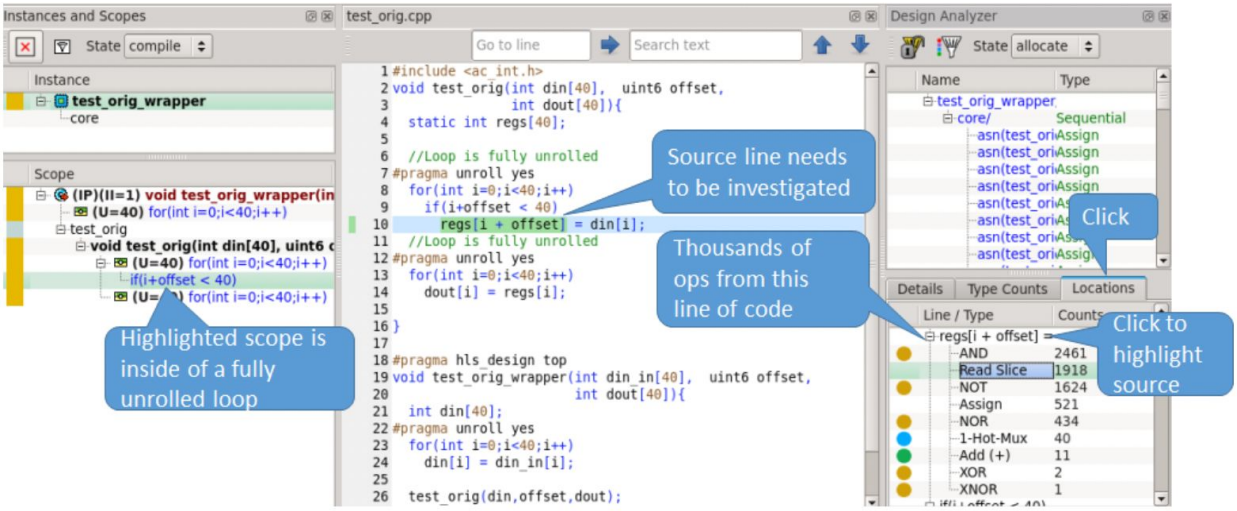


Figure 28. Detail of the variable "*regs[i+offset]*" in file "*test_orig.cpp*".

11. Change the Design Analyzer state to "*Memories*". And reanalyze the design by checking the Locations tabs. The "memories" state is after memory optimizations including register splitting so analyzing here still provides all of the necessary detail. This is very useful since more complicated design may get stuck in allocate.

12. Close Design Analyzer

13. Go to **File** → **Run** script and select "*coding_style.tcl*" file. This can take a couple of minutes to complete. This will make "*test.cpp*" the top-level design and will exclude "*test_orig.cpp*" from synthesis but still use it to compare against in the testbench. The script will also synthesize the design.

14. Open "*test.cpp*" and note the following:

a. An additional loop with index "*j*" has been added to compute the offset. Because the "*j*" loop is fully unrolled the loop iterator "*j*" will become a constant allowing Catapult to optimize the design.

b. "*Regs*" is indexed by *"i+j"* which resolves to constants because both loops are fully unrolled.

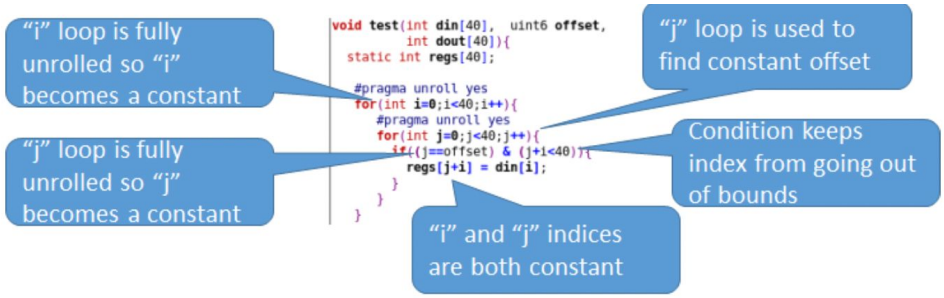
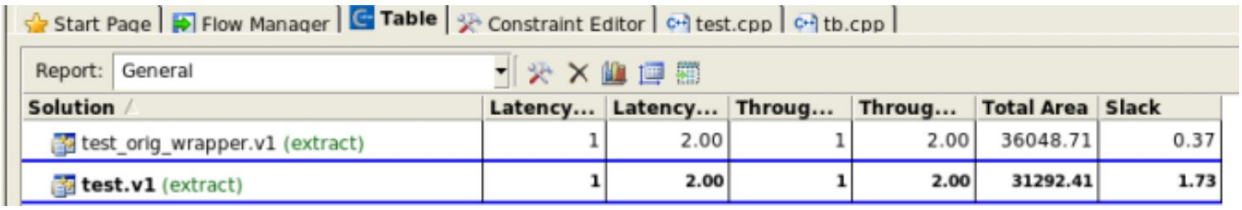


Figure 29. Detail of the new loop added in file "*test.cpp*".

15. Look at the Table View and note the area improvement.



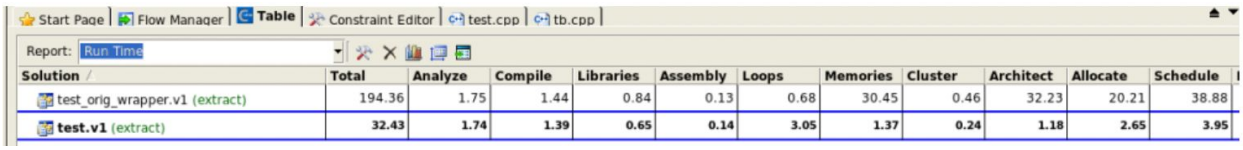
Start Page | Flow Manager | Table | Constraint Editor | test.cpp | tb.cpp

Report: General

Solution	Latency...	Latency...	Throug...	Throug...	Total Area	Slack
test_orig_wrapper.v1 (extract)	1	2.00	1	2.00	36048.71	0.37
test.v1 (extract)	**1**	**2.00**	**1**	**2.00**	**31292.41**	**1.73**

Figure 30. Table View detail for the newly synthesized sources.

16. Change the Table View to show Runtime and note the runtime improvement.



Start Page | Flow Manager | Table | Constraint Editor | test.cpp | tb.cpp

Report: Run Time

Solution	Total	Analyze	Compile	Libraries	Assembly	Loops	Memories	Cluster	Architect	Allocate	Schedule
test_orig_wrapper.v1 (extract)	194.36	1.75	1.44	0.84	0.13	0.68	30.45	0.46	32.23	20.21	38.88
test.v1 (extract)	**32.43**	**1.74**	**1.39**	**0.65**	**0.14**	**3.05**	**1.37**	**0.24**	**1.18**	**2.65**	**3.95**

Figure 31. Table View detail (Runtime-sorted) for the newly synthesized sources.

17. Set the filter for warnings back to Show All.

18. Go the **Project Files** → **Verification** folder and run the C++ design and testbench in g++. The testbench compares the original design against the modified code.

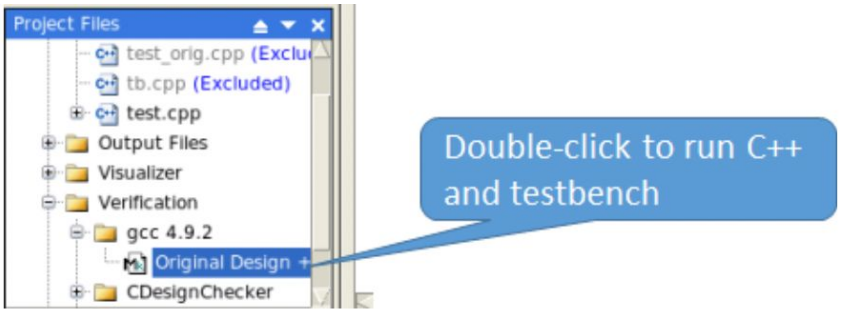


Figure 32. Run C++ design of testbench to verify synthesis.

19. Look at the transcript and you should see the testbench reports 0 errors.

20. Close Catapult.

References

[1] Learn makefiles online content. https://makefiletutorial.com/